

TECHNICAL ENGINEERING

Arquitectura de Software

Documento Oficial de Arquitectura v2.0

1. Resumen Ejecutivo y Alcance

El presente documento describe la arquitectura técnica, las decisiones de diseño y las tecnologías aplicadas para el desarrollo del producto **MediSystem**. Este sistema ha sido diseñado como una solución integral para la gestión de consultorios médicos, resolviendo el problema de agendamiento, control de pacientes, historiales y la aplicación de reglas comerciales (como la promoción de fidelización 1:3).

Objetivo Principal: Proveer un sistema altamente escalable, seguro y auditable que separe las preocupaciones de presentación (Frontend) y lógica de negocio (Backend) permitiendo futuras expansiones como aplicaciones móviles o integraciones con aseguradoras.

2. Patrón Arquitectónico

MediSystem adopta una **Arquitectura Cliente-Servidor (Tiered Architecture)** y, a nivel de backend, utiliza un patrón **Modelo-Vista-Controlador (MVC) adaptado para APIs RESTful**. Las capas están estrictamente delimitadas, promoviendo la cohesión y reduciendo el acoplamiento.

CAPA DE SISTEMA	STACK TECNOLÓGICO	RESPONSABILIDAD Y FUNCIÓN
Frontend (Cliente)	Next.js React	Renderizado de UI, gestión del estado del usuario, UX/UI, enrutamiento en navegador.
API Gateway	Node.js Express	Recepción de peticiones HTTP, parseo JSON, sanitización, manejo de CORS.

CAPA DE SISTEMA	STACK TECNOLÓGICO	RESPONSABILIDAD Y FUNCIÓN
Seguridad	JWT Bcrypt	Autenticación stateless, autorización (RBAC), encriptación de passwords.
Lógica de Negocio	TypeScript Jest	Reglas comerciales (Citas 1:3), validaciones de dominio.
Persistencia	Prisma ORM MySQL	Abstracción de base de datos, mitigación de SQL Injection.

3. Diagrama Lógico de Interacción

```

[ Cliente React ] ----> (HTTP REST) ----> [ Nginx Proxy ]
                                     |
                                     [ Express.js API ]
                               /      |      \
          [Auth JWT] [Citas] [Pacientes]
                               |
                               [ Prisma ORM ]
                               |
                               [( MySQL DB )]
```

4. Lógica de Negocio (Regla 1:3)

Uno de los core-components del sistema es el motor de agendamiento. Por requerimiento directivo, se ha programado un algoritmo que evalúa la carga de pacientes de un médico. El sistema cuenta cuántas citas pagadas ("cobro") ha atendido el doctor en el ciclo actual. Si `count % 3 === 0`, la próxima cita que se

inserte en la base de datos se etiquetará automáticamente con el flag `isFree = true`.